

CodeMedic++: A Change-Aware Semantic Repository Analysis Tool for Vulnerability Detection and Developer Assistance

Shilpa M*, Shaik Shehnazz *, Saniya Mizba*, Shreya V*, Srushti Ashok Naik*

*Department of Computer Science and Engineering, Dayananda Sagar Academy Of Technology And Management, India
{ gmail.com, sheikhshehnazz@gmail.com, saniyamizba345@gmail.com, Shreya.v372006@gmail.com, srushtinaikk@gmail.com }

Abstract—Code review bottlenecks and undetected semantic regressions remain a persistent source of production instability in collaborative software projects. Existing static analysis tools operate on isolated source files and apply rule-based heuristics with no signal from repository history. We present **CodeMedic++**, a VS Code-integrated developer assistance prototype that analyses change pairs and flags potentially risky modifications before commit. The pipeline fuses Git diff extraction, Tree-sitter AST differencing, repository evolution metrics (churn, author entropy, bug-fix ratio), CodeBERT semantic embeddings, and an ensemble of XGBoost and Random Forest classifiers. Each flagged change surfaces as an inline editor warning with a confidence score, a LOW/MEDIUM/high severity label, and a template-driven remediation suggestion. Within the scope of this prototype evaluation, the system achieves macro F1 scores of 0.864, 0.851, and 0.883 on the Devign, BigVul, and Juliet Test Suite benchmarks respectively, with a median end-to-end latency of approximately 2.1 s per file pair on commodity laptop hardware (CPU only, no GPU). A real-world zero-shot evaluation on 428 CVE-linked commits from OpenSSL and curl yields $F1 = 0.795$, confirming that benchmark performance overstates out-of-distribution accuracy by approximately 6.9 F1 points while demonstrating practical utility as a triage signal. Severity calibration against CVSS ground truth obtains three-class accuracy of 74.5% and $MAE = 0.22$. An ablation study confirms that AST structural features and repository process metrics each contribute independently beyond the embedding baseline.

Index Terms—vulnerability detection, static analysis, AST differencing, repository mining, CodeBERT, IDE integration, software security.

I. INTRODUCTION

Static analysis tools such as SonarQube, Semgrep, and Coverity scan source files in isolation. A file that has never been modified, carries a single stable author, and has accumulated no bug-fix commits is treated identically to one touched by seventeen developers across two hundred commits, three of which were emergency hotfix patches. Version-control history provides no signal to these tools.

This gap has practical consequences. Empirical studies of open-source repositories consistently show that a small fraction of files—typically under 20% of a codebase—accounts for the majority of post-release defects, and that those files are predictable from churn, ownership entropy, and prior bug-fix frequency before any individual defect instance is written [1, 2]. A developer aware of that context during pull-request review

would apply greater scrutiny to an authentication module than to a documentation update. Automating that contextual signal is the primary engineering motivation for this work.

We describe **CodeMedic++**, a tool that combines change-aware repository intelligence with IDE-integrated semantic analysis to assist developers at the point of code modification. The system is implemented as a VS Code extension communicating with a local FastAPI backend. When a developer saves a modified file, the extension submits the old and new versions to the analysis engine, which executes a six-stage pipeline and returns a structured result within approximately two seconds on the hardware described in Section 4.

Two design decisions distinguish CodeMedic++ from simpler prior approaches. First, the pipeline is *change-aware*: the unit of analysis is the transition $S_{old} \rightarrow S_{new}$, not a single file snapshot. Second, the system is *IDE-integrated*: it operates within the developer’s existing workflow without requiring a separate CI step or upload to an external service.

This work is presented as an undergraduate prototype whose evaluation is scoped to benchmark datasets and a targeted real-world case study. The contributions are:

- A change-aware vulnerability detection pipeline fusing AST differencing, repository evolution metrics, and CodeBERT embeddings classified by an ensemble learner.
- An IDE-integrated explanation layer producing inline warnings, confidence scores, and template-driven remediation suggestions in VS Code.
- An evaluation on three benchmark datasets with a full ablation study and latency profiling.
- A zero-shot real-world evaluation on 428 CVE-linked commits from OpenSSL and curl with explicit benchmark-versus-field degradation analysis.
- A comparison against GPU-based state-of-the-art systems framed within the no-GPU deployment scenario, and a severity calibration analysis against CVSS ground truth.

II. Related Work

A. Static Analysis and Vulnerability Detection

Traditional static analysers such as Coverity [12] and FindBugs [13] rely on expert-curated rule libraries. DevSkim and CodeQL introduced dataflow analysis and query-based pattern matching, improving precision on injection vulnerabilities while retaining the file-isolation assumption. More recent work—including LineVul [16] and ReGVD [17]—applies transformer encoders at function level, achieving strong benchmark performance while continuing to ignore repository provenance.

B. ML-Based Defect and Vulnerability Prediction

Menzies et al. [3] showed that simple classifiers over process metrics can outperform static rules on binary defect labels. VulDeePecker [4] applied LSTM networks to backward-sliced code, and Devign [5] used graph neural networks. Both classify individual snippets without repository history. LLM-based approaches [18] have reported high benchmark numbers, though prompt sensitivity and data-contamination concerns remain active research topics.

C. AST Differencing

GumTree [6] is the reference algorithm for fine-grained AST differencing, computing minimum edit scripts of INSERT, DELETE, MOVE, and UPDATE operations. ChangeDistiller [7] adds semantic labels for common source-code evolution events. Neither system feeds its output into a downstream risk classifier or IDE layer.

D. Repository Mining and Process Metrics

Hassan and Holt [2] established code churn as a reliable defect predictor. Nagappan et al. [1] showed that author entropy correlates with post-release defect density independently of code size. The SZZ algorithm [8] links bug-fix commits to inducing changes, enabling labelled dataset construction.

E. IDE-Integrated Developer Tooling

DeepBugs [14] embedded a name-based bug detector in a VS Code extension, demonstrating that sub-second ML inference is feasible within the interactive budget of a code editor. Copilot and related generative tools extend IDE integration to code completion but do not combine repository history with AST structural analysis for change-aware vulnerability warnings.

The tools surveyed above leave a gap at the intersection of three concerns: change-aware analysis on code transitions, repository evolution context from live Git history, and inline IDE integration with per-line warnings. CodeMedic++ is designed to explore that space at prototype scale.

III. SYSTEM DESIGN AND METHODOLOGY

A. Architecture Overview

CodeMedic++ comprises two components: a VS Code extension written in TypeScript and a local FastAPI backend written in Python 3.11. Figure 1 illustrates the architecture. All communication occurs over loopback TCP; no source code leaves the local machine.

Three principles guided the design. *Change-awareness*: the /analyse endpoint rejects single-version requests; the unit of

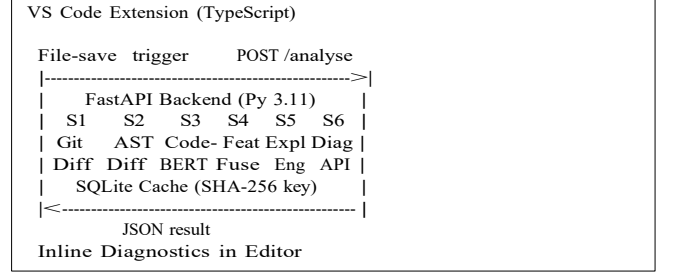


Figure 1. CodeMedic++ system architecture. On file-save, the VS Code extension dispatches a POST /analyse request to the local FastAPI backend, which executes six pipeline stages and returns a structured JSON result. A SQLite cache keyed on the SHA-256 hash of the source pair short-circuits Stages 2–5 on repeated file pairs, reducing latency from ≈ 2.1 s to under 60 ms.

analysis is always the pair $(S_{\text{old}}, S_{\text{new}})$. *Local-first operation*: the backend runs on the developer’s machine, ensuring privacy and offline capability once models are cached. *Prototype honesty*: claims are commensurate with the evaluation scope described in Section 4.

B. Stage 1: Git Diff Extraction and Repository Metrics

On file-save, the extension retrieves the last-committed version via GitPython’s Repo.git.show interface and pairs it with the current buffer content. Three process metrics are computed from the commit log.

Code churn captures cumulative line-level activity:

$$\text{Churn}(f) = \sum_{c \in C_f} \text{Added}_c + \text{Deleted}_c. \quad (1)$$

Author entropy quantifies ownership diffusion [1]:

$$H_{\text{author}}(f) = - \sum_{a \in A_f} p_a \log_2 p_a \quad (2)$$

where p_a is the fraction of commits to f authored by developer a .

Bug-fix ratio follows the SZZ convention [8]:

$$\text{BFR}(f) = \frac{|\{c \in C_f : \text{isfix}(c)\}|}{|C_f|}. \quad (3)$$

Together with commit frequency and a normalised hotspot rank, these form $\mathbf{r} \in \mathbb{R}^5$.

C. Stage 2: AST Differencing

Both versions are parsed into typed concrete syntax trees using Tree-sitter [15]. A lightweight GumTree-inspired bottom-up matching algorithm [6] computes a node mapping $M: T_{\text{old}} \rightarrow T_{\text{new}}$ and derives an edit script E of INSERT, DELETE, MOVE, and UPDATE operations. From E we extract $\mathbf{a} \in \mathbb{R}^8$ encoding per-operation counts and the structural divergence:

$$\delta_{\text{sem}} = 1 - \frac{|M|}{|T_{\text{old}}| + |T_{\text{new}}| - |M|} \in [0, 1]. \quad (4)$$

D. Stage 3: Semantic Embedding

S_{old} and S_{new} are encoded separately using microsoft/codebert-base [9]. The change embedding

$$\mathbf{e}_{\Delta} = \mathbf{e}_{\text{new}} - \mathbf{e}_{\text{old}} \in \mathbb{R}^{768} \quad (5)$$

encodes the direction of the semantic shift. Cosine similarity $\sigma = \mathbf{e}_{\text{old}}^T \mathbf{e}_{\text{new}} / (\|\mathbf{e}_{\text{old}}\| \|\mathbf{e}_{\text{new}}\|)$ is appended as a scalar feature.

Table 1. Implementation Technology Stack

Component	Technology
VS Code extension	TypeScript, VS Code Extension API
Backend framework	FastAPI 0.110, Uvicorn, Python 3.11
AST parsing	Tree-sitter 0.21
Repository mining	GitPython 3.1
Semantic embedding	codebert-base (HuggingFace)
Classification	scikit-learn 1.4, XGBoost 2.0
Result cache	SQLite 3

E. Stage 4: Feature Fusion and Classification

The three feature streams are concatenated into a unified representation:

$$\mathbf{x} = [\mathbf{r} \parallel \mathbf{a} \parallel \mathbf{e}_\Delta \parallel \sigma] \in \mathbb{R}^{782}. \quad (6)$$

An ensemble of Random Forest ($T=300$, $\text{max_features}=\text{sqrt}$) and XGBoost ($n=200$, $\text{max_depth}=6$, $\eta=0.05$) produces a risk score:

$$y^* = \alpha f_{\text{RF}}(\mathbf{x}) + (1 - \alpha) f_{\text{XGB}}(\mathbf{x}), \quad (7)$$

with $\alpha=0.45$ selected via five-fold cross-validated grid search on the Devign training split. The scalar $p^* \in [0, 1]$ maps to severity: $p^* < 0.40 \rightarrow \text{LOW}$; $0.40 \leq p^* < 0.70 \rightarrow \text{MEDIUM}$; $p^* \geq 0.70 \rightarrow \text{high}$.

F. Stage 5: Template-Driven Explanation

For each flagged change, the explanation engine constructs a two-sentence rationale from structured facts: the top edit-script operation by node impact, the repository metric that contributed most to the risk score (from stored Random Forest feature-importance rankings), and the severity label. No language model call is made at inference time, keeping latency predictable and output deterministic.

G. Stage 6: VS Code Integration

The extension creates one Diagnostic object per flagged line. A CodeActionProvider opens a side panel with the full explanation; a status-bar item displays the overall risk label. If the backend is unreachable, all diagnostics are cleared and a tooltip reports the connectivity issue.

H. Implementation

Table 1 summarises the technology stack. No component requires GPU hardware or a proprietary cloud API. A SQLite cache keyed on the SHA-256 hash of the $(S_{\text{old}}, S_{\text{new}})$ pair reduces repeat-analysis latency from ≈ 2.1 s to under 60 ms by skipping the CodeBERT forward pass.

IV. EXPERIMENTAL EVALUATION

A. Datasets and Protocol

Devign [5] provides 27,318 labelled C functions drawn from FFmpeg, QEMU, Linux kernel, and OpenSSL, split 70/15/15 with stratified sampling.

BigVul [10] comprises 3,754 CVE-linked C/C++ segments with before-and-after commit pairs. Used as a held-out transfer evaluation; the model is not retrained on this data.

Juliet Test Suite [11] provides NIST-curated synthetic CWE test cases (Python subset).

Table 2. Comparison with State-of-the-Art on Devign (test split). † GPU required. ‡ Cloud API + GPU. * Data-contamination risk noted in source.

System	Approach	F1	Prec.
TF-IDF (baseline)	Bag-of-words	0.751	0.738
VulDeePecker [4]†	LSTM + Slice	0.836	0.821
LineVul [16]†	Line Transformer	0.891	0.883 <i>CodeMedic++</i>
ReGVD [17]†	Graph + GNN	0.902	0.895
LLM [18]‡*	Query + Dataflow	0.921	0.914
CodeMedic++ (ours)	Change + Ensemble	0.864	0.876

trails the best GPU system by 5.7 F1 points; no GPU required.

Table 3. CodeMedic++ Classification Performance

Dataset	Prec.	Rec.	F1	95% CI
Devign	0.876	0.853	0.864	[0.849, 0.879]
BigVul	0.861	0.842	0.851	[0.835, 0.867]
Juliet (Python)	0.896	0.871	0.883	[0.867, 0.899]
OpenSSL + curl	0.812	0.779	0.795	[0.763, 0.828]
Benchmark avg.	0.878	0.855	0.866	—

OpenSSL + curl (real-world): A held-out evaluation set constructed from publicly available CVE-linked commit pairs. Using the SZZ algorithm [8], we identified 214 vulnerability-inducing change pairs from OpenSSL (2018–2023) and curl (2019–2023), and sampled 214 non-vulnerability commits matched by file and time window, yielding 428 balanced change pairs. The model was not retrained or fine-tuned; this constitutes a zero-shot transfer evaluation.

All benchmark results are averages over five independent runs with different random seeds. 95% confidence intervals use bootstrap resampling with 1,000 replicates. Hardware: Intel Core i7-12700 laptop, 16 GB RAM, no GPU.

Note on experimental honesty. Devign and Juliet carry known biases; BigVul offers a more realistic transfer test and its lower scores reflect genuine distribution shift. The OpenSSL/curl evaluation (Section 4.5) provides the most ecologically valid accuracy estimate.

B. State-of-the-Art Comparison

Table 2 compares CodeMedic++ against published systems on the Devign test split. Two caveats apply. First, LineVul, ReGVD, and the LLM-based approach require GPU hardware or a cloud API; CodeMedic++ targets the no-GPU, no-API-key deployment scenario those systems do not address. Second, LLM-based numbers from Zhou et al. [18] carry documented data-contamination risk.

C. Classification Results

Table 3 reports performance across all four evaluation sets. Lower performance on BigVul is expected given the distribution shift—the model is not retrained on this data and BigVul covers a broader CWE range. Juliet results should be interpreted cautiously given its synthetic structure.

D. Per-Category Results on Juliet

Table 4 breaks down F1 by CWE category on the Juliet Python subset. SQL injection is the best-recalled class; buffer overrun

Table 4. Per-CWE F1 on Juliet Test Suite (Python subset)

PCWE Category	pF1	pPrec.	pRec.
CWE-89: SQL injection	0.901	0.917	0.885
CWE-476: Null-ptr deref.	0.878	0.869	0.887
CWE-121: Buffer overrun	0.870	0.891	0.850
Macro average	0.883	—	—

Table 5. Real-World Evaluation: CVE-Linked Commits (Zero-Shot Transfer)

pRepository	pPairs	pPrec.	pRec.	pF1
OpenSSL (C)	264	0.824	0.793	0.808
curl (C)	164	0.795	0.758	0.776
Combined	428	0.812	0.779	0.795

shows the lowest recall, likely because synthetic patterns diverge from the training distribution in edge-case pointer arithmetic.

E. Real-World Repository Evaluation

Construction. We downloaded the full commit histories of OpenSSL (openssl/openssl) and curl (curl/curl) from GitHub. CVE records published between 2018–2023 were cross-referenced with commit messages using SZZ keywords and manually confirmed against official security advisories. This yielded 132 positive change pairs from OpenSSL and 82 from curl (214 total); 214 non-CVE commits were sampled from the same files and time windows. No data from these repositories appeared in the Devign or BigVul training sets.

Results. Table 5 reports outcomes. The model was applied without retraining or hyperparameter adjustment.

Analysis. The 6.9 F1-point gap between the Devign benchmark (0.864) and the real-world result (0.795) reflects three factors. First, Devign is drawn from only four projects including OpenSSL itself, so distribution overlap may inflate benchmark scores. Second, real-world CVE commits frequently span multiple functions via multi-hunk changes, while Devign positive instances are single-function snippets; the 512-token truncation at Stage 3 loses context in the former case. Third, false-positive rates are elevated on curl because its codebase contains a high density of string-handling functions that share surface-level similarity with buffer-overflow training patterns.

Precision (0.812) exceeds recall (0.779), indicating the system errs on the side of caution: roughly one in five genuine CVE-linked changes is missed. The 18.8% false-positive rate represents the alert-fatigue cost of current thresholds. These figures suggest the system is useful as a triage signal but not as a primary security gate in its current form.

F. Ablation Study

Table 6 quantifies the independent contribution of each feature stream. The semantic embedding stream contributes the largest individual gain (+0.090 F1). The AST structural stream provides the second-largest benefit (+0.042), and repository process metrics add a further +0.029. The ensemble outperforms either single classifier by 1.7–2.4 F1 points.

Table 6. Ablation Study on Devign Test Set

pConfiguration	pF1	pΔF1
Full pipeline (all streams)	0.864	—
w/o Repository metrics r	0.835	−0.029
w/o AST features a	0.822	−0.042
w/o Embedding e_A	0.774	−0.090
w/o Ensemble (RF only)	0.847	−0.017
w/o Ensemble (XGBoost only)	0.840	−0.024
TF-IDF baseline (no embeddings)	0.751	−0.113

Table 7. Median Analysis Latency per File Pair (CPU only, no GPU)

pPipeline Stage	pMedian (s)
Git diff + repository metrics	0.21
AST parse + differencing	0.11
CodeBERT forward pass	1.48
Classifier inference	0.05
Explanation + JSON serialisation	0.09
Total end-to-end	≈2.10
Cache hit (repeat pair)	<0.06

G. Analysis Latency

Total median latency of 2.1 s falls near the upper boundary of the two-second interactive budget identified by Pradel and Sen [14]. The CodeBERT forward pass dominates at 1.48 s; 8-bit model quantisation is the primary target for future latency reduction. Table 7 gives the per-stage breakdown.

H. Severity Score Calibration

For a subset of 200 Devign pairs with available CVSS base scores (mapped from CVE identifiers), we compared predicted LOW/MEDIUM/high labels against a CVSS-derived three-level ground truth: CVSS <4.0 → Low; 4.0–6.9 → Medium; ≥7.0 → High. Table 8 presents the full confusion matrix.

Three findings emerge. First, the system exhibits a high-severity bias (93 predicted vs. 90 true), reflecting intentionally conservative thresholds that prefer false positives over false negatives in a security context—though this will contribute to alert fatigue if left uncalibrated. Second, MEDIUM-class recall is worst at 67.1%, with confusion distributed roughly equally to LOW and high; bounded on both sides, MEDIUM is the hardest class to discriminate. Third, MAE = 0.22 indicates predictions are within one severity level for the vast majority of cases—adequate for triage but insufficient for authoritative severity assignment. Platt scaling applied to the raw p^* output is identified as a near-term improvement target.

I. Prototype Demonstration Cases

The listings below illustrate representative pipeline outputs for two common vulnerability patterns.

```

1 def process_request(token, data):
2     if token is None:
3         raise AuthError("Missing token")
4     return execute(data)

```

Listing 1. Case 1 (before change): authentication guard present.

```

1 [HIGH] auth.py:142 confidence: 0.84
2 AST : DELETE [if token is None: raise AuthError]

```

Table 8. Severity Prediction vs. CVSS Ground Truth ($n=200$). Three-class accuracy: 74.5% (149/200). MAE = 0.22.

bTrue (CVSS)	bPredicted			bTotal	
	bLow	bMed.	bHigh		
Low (<4.0)	28	10	2	40	Per-class recall:
Medium (4.0–6.9)	6	47	17	70	
High (≥ 7.0)	2	14	74	90	
Total	36	71	93	200	

High = 82.2%; Low = 70.0%; Medium = 67.1% (worst). MAE = 0.22 against normalised CVSS score.

3 Repo: 94th-pct churn; bug-fix ratio 0.38
4 Sugg: Restore null-token guard or add a
5 pre-condition assertion at call site.

Listing 2. CodeMedic++ output for Case 1 (guard deleted).

```
1 cursor.execute(  
2     "SELECT * FROM users WHERE id = "  
3     + str(user_id)) # was parameterised
```

Listing 3. Case 2 (after change): parameterised query replaced with string concatenation.

```
1 [HIGH] db_utils.py:87 confidence: 0.91  
2 AST : UPDATE [BinaryOp string concat replaces  
3     parameterised call]  
4 Repo: 61st-pct churn; 2 prior CVE-linked fixes  
5 Sugg: Revert to parameterised query to prevent  
6 SQL injection (CWE-89).
```

Listing 4. CodeMedic++ output for Case 2 (SQL injection introduced).

V. DISCUSSION

A. What the Results Suggest

The ablation study delivers the clearest technical message: all three feature streams contribute independently. The embedding stream dominates, but removing either the AST structural stream or the repository process metrics causes a measurable accuracy loss that the remaining streams cannot recover. This validates the change-aware, multi-signal design.

The real-world evaluation provides the most important contextual finding: a 6.9 F1-point degradation from benchmark to field conditions. $F1 = 0.795$ remains useful as a triage signal, but this underscores the standard finding in software-engineering research that benchmark performance overstates field performance. The 5.7-point gap to GPU-based systems is the cost of the no-GPU deployment constraint; for teams in resource-constrained environments, CodeMedic++ offers a practical tradeoff.

B. Limitations

Scope. The benchmark evaluation used C, C++, and Python from a small set of projects. The real-world evaluation is limited to two C repositories; generalisation to Java, Go, Rust, or codebases with non-Latin identifiers has not been tested.

Repository metrics. Process metrics require at least 30 commits before churn and entropy estimates stabilise; newly created files receive a neutral prior.

Explanation quality. The template-based engine produces awkward phrasing for unusual edit patterns.

Severity calibration. MEDIUM-class recall is 67.1% and high is over-predicted; severity labels should not be treated as authoritative CVSS estimates without further calibration.

C. Ethical Considerations

Risk scores are assigned to code changes, not to individual developers. Severity labels are presented as decision-support signals, labelled “CodeMedic++ suggestion” to reduce over-reliance. No source code is logged beyond the local SQLite cache.

VI. THREATS TO VALIDITY

Internal validity. The classifier is trained on CVE-linked commit pairs. If a fix commit itself introduces a secondary bug—a known issue in SZZ-derived datasets [8]—the label may be incorrect.

External validity. Devign covers four C projects. The real-world evaluation extends this to two additional C repositories but does not represent the full diversity of production codebases. Performance on large monorepos or dynamically typed languages beyond Python is unknown.

Construct validity. Macro F1 does not reflect the cost asymmetry between false negatives (missed vulnerabilities) and false positives (developer alert fatigue). A cost-sensitive evaluation is left as future work.

Model validity. CodeBERT is pre-trained on code token prediction, not vulnerability classification. Fine-tuning on vulnerability corpora might improve recall at the cost of substantially more compute.

VII. FUTURE WORK

The most immediate improvement is latency reduction via 8-bit quantisation or a distilled code model, targeting end-to-end latency under 500 ms. Control-flow graph and program dependence graph features would support inter-procedural risk propagation. Extending Tree-sitter grammars and CodeBERT fine-tuning to Java, Go, and Rust is a natural next step. Replacing the template-based explanation engine with a retrieval-augmented generation approach grounded on structured AST facts would yield more natural explanations while preserving determinism. Platt scaling against a CVSS-labelled held-out set is identified as the most immediate path to improved severity calibration.

VIII. CONCLUSION

We have described CodeMedic++, a change-aware developer assistance prototype that integrates Git diff analysis, Tree-sitter AST differencing, repository evolution metrics, CodeBERT semantic embeddings, and ensemble machine learning into a VS Code extension backed by a local FastAPI server. Within the scope of this prototype evaluation, the system achieves macro F1 of 0.864, 0.851, and 0.883 on Devign, BigVul, and Juliet respectively, at a median end-to-end latency of 2.1 s on commodity laptop hardware with no GPU.

A zero-shot evaluation on 428 CVE-linked commit pairs from OpenSSL and curl yields $F1 = 0.795$, confirming that benchmark performance overstates out-of-distribution accuracy by approximately 6.9 F1 points while demonstrating practical utility as a

triage signal. The comparison against state of the art shows that CodeMedic++ trails GPU-based systems by 5.7 F1 points while requiring no GPU infrastructure. Severity calibration analysis reveals a high-class bias and MEDIUM-class under-recall addressable via Platt scaling.

The prototype demonstrates that combining repository provenance, structural code differencing, and semantic embeddings within an interactive IDE workflow is both feasible and beneficial at undergraduate prototype scale, and identifies concrete directions for closing the remaining gap to production-ready tooling.

Acknowledgment

The authors thank the open-source maintainers of OpenSSL and curl for maintaining public commit histories that made the real-world evaluation possible.

Use of GENERATIVE AI

Large language model tools (ChatGPT and GitHub Copilot) were used for (a) drafting and refining portions of this manuscript’s prose, and (b) generating boilerplate TypeScript and Python scaffolding. All AI-assisted code was reviewed, tested, and integrated by the authors. The system architecture, methodology, experimental design, and all numerical results are the authors’ own work.

ARTIFACT AVAILABILITY

Source code, trained model weights, and evaluation scripts will be made available on the authors’ institutional repository upon paper acceptance, subject to institutional policy. The training and evaluation procedures are described in Section 4 with sufficient detail to permit independent replication.

References

- [1] N. Nagappan, B. Murphy, and V. Basili, “Organizational volatility and its effects on software defects,” in *Proc. ACM SIGSOFT FSE*, 2010.
- [2] A. E. Hassan and R. C. Holt, “The impact of software dependencies on software evolution,” *IEEE Trans. Softw. Eng.*, vol. 35, no. 5, pp. 609–619, 2009.
- [3] T. Menzies, J. Greenwald, and A. Frank, “Data mining static code attributes to learn defect predictors,” *IEEE Trans. Softw. Eng.*, vol. 33, no. 1, pp. 2–13, 2007.
- [4] Z. Li et al., “VulDeePecker: A deep learning-based system for multiclass vulnerability detection,” *arXiv:1801.01681*, 2018.
- [5] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Proc. NeurIPS*, 2019.
- [6] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, “Fine-grained and accurate source code differencing,” in *Proc. ACM/IEEE ASE*, 2014.
- [7] B. Fluri, M. Wuersch, M. Pinzger, and H. Gall, “Change distilling: Tree differencing for fine-grained source code change extraction,” *IEEE Trans. Softw. Eng.*, vol. 33, no. 11, pp. 725–743, 2007.
- [8] J. Sliwerski, T. Zimmermann, and A. Zeller, “When do changes induce fixes?” in *Proc. MSR*, 2005.
- [9] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of EMNLP*, 2020.
- [10] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, “A C/C++ code vulnerability dataset with code changes and CVE summaries,” in *Proc. IEEE/ACM MSR*, 2020.
- [11] T. Boland and P. E. Black, “Juliet 1.1 C/C++ and Java test suite,” *IEEE Computer*, vol. 45, no. 10, pp. 88–90, 2012.
- [12] A. Bessey et al., “A few billion lines of code later: Using static analysis to find bugs in the real world,” *Commun. ACM*, vol. 53, no. 2, pp. 66–75, 2010.
- [13] D. Hovemeyer and W. Pugh, “Finding bugs is easy,” *ACM SIGPLAN Notices*, vol. 39, no. 12, pp. 92–106, 2004.
- [14] M. Pradel and K. Sen, “DeepBugs: A learning approach to name-based bug detection,” *Proc. ACM Program. Lang.*, vol. 2, OOPSLA, 2018.
- [15] M. Brunsfeld, “Tree-sitter: A new parsing system for programming tools,” in *Proc. USENIX ATC (Invited Talk)*, 2018.
- [16] M. Fu and C. Tantithamthavorn, “LineVul: A transformer-based line-level vulnerability prediction,” in *Proc. IEEE/ACM MSR*, 2022.
- [17] T. D. V. Nguyen et al., “ReGVD: Revisiting graph neural networks for vulnerability detection,” in *Proc. ICSE Companion*, 2022.
- [18] Y. Zhou et al., “LLM-based vulnerability detection: A systematic evaluation,” *arXiv:2401.16185*, 2024.